



Ingeniería en Ciencias de la Computación

Sistemas Distribuidos

Empaquetamiento y Despliegue de NoticeBoard con Docker

Desarrollado por:

Jose Salamea

Cuenca, Ecuador
23 de mayo de 2026

Índice

1. Cuestionario de Investigación y Diagnóstico Técnico	2
2. Diseño del Dockerfile	3
2.1. Archivo de Exclusiones (.dockerignore)	3
2.2. Implementación del Dockerfile	3
2.3. Decisiones de Diseño Justificadas	4
2.4. Cuestionario de Conceptos de Contenedores	4
3. Construcción y Verificación	5
3.1. Bitácora de Construcción y Verificación Local	5
3.2. Evidencias de Verificación Funcional	5
3.3. Análisis Técnico de la Imagen (Capas y Dimensiones)	6
3.4. Comando para Despliegue en Nuevos Entornos	6
4. Publicación	6
4.1. Enlace al Repositorio Público	7
4.2. Verificación del Despliegue desde el Registry	7
4.3. Análisis de Versionado y Seguridad en Producción	7
5. Uso de Inteligencia Artificial	7

1. Cuestionario de Investigación y Diagnóstico Técnico

1. **¿Qué versión exacta de Python requiere la aplicación? ¿Dónde lo encontró?**

Se encontró la versión exacta dentro del archivo `README.md`, (Python 3.11 o superior).

2. **Revisión de dependencias (`requirements.txt`). Por cada dependencia, explique en una oración qué hace dentro de la aplicación (en el contexto de NoticeBoard):**

- **Flask (v3.0.3):** Encargado de gestionar las peticiones HTTP, el enrutamiento de las URLs y el renderizado de la interfaz.
- **Flask-SQLAlchemy (v3.1.1):** Actúa facilitando la creación, lectura y actualización de registros en la base de datos SQLite sin escribir código SQL nativo.

3. **La aplicación usa SQLite. ¿Qué implica eso para la persistencia de datos cuando el contenedor se destruye y se vuelve a crear? ¿Cómo resolvería ese problema en producción?**

Quiere decir que el almacenamiento es efímero. Al ser SQLite una base de datos basada en un archivo local dentro del contenedor, si se borra la instancia elimina dicho archivo, provocando la pérdida total e irreversible de los anuncios guardados.

En producción, este problema se resuelve mediante dos alternativas:

- **Volúmenes de Docker:** Mapear el directorio o archivo de la base de datos hacia el almacenamiento persistente del host, independiente al ciclo de vida del contenedor.
- **Base de datos externa:** Migrar el backend de SQLite a un motor dedicado como PostgreSQL o MySQL. Ejecutando en un servidor independiente, inyectando la cadena de conexión por variables de entorno.

4. **¿En qué puerto escucha la aplicación? ¿Es un valor fijo en el código o es configurable? ¿Cómo lo configuraría desde fuera del contenedor sin modificar el código fuente?**

Por defecto, la aplicación escucha en el puerto 5000. No es un valor fijo; podemos observar en la línea 15 de `config.py`, ya está diseñada para ser **configurable dinámicamente** mediante la instrucción `os.environ.get("PORT", 5000)`.

Para cambiar este valor desde fuera del contenedor sin alterar el código fuente, se debe incluir la variable de entorno `PORT` en el comando de ejecución de Docker utilizando la bandera `-e`, mapeando simultáneamente el nuevo puerto elegido:

```
1 docker run -p 8080:8080 -e PORT=8080 miusuario/noticeboard :1.0.0
```

5. **La aplicación tiene archivos estáticos (CSS, imágenes, JavaScript). En producción, ¿es una buena práctica que Flask sirva esos archivos directamente? ¿Qué alternativa existe y por qué se usa?**

No es una buena práctica. Tras inspeccionar la estructura del proyecto, se verificó la existencia del archivo `style.css` dentro del directorio `app/static/`.

El servidor Flask está diseñado exclusivamente para desarrollo y procesa peticiones de forma síncrona; obligarlo a servir recursos estáticos consume ciclos de CPU e hilos de ejecución que deberían reservarse para la lógica de backend en Python.

La alternativa estándar en entornos de producción es delegar esta tarea a un proxy inverso como **Nginx**. Estas herramientas están optimizadas a nivel de kernel para despachar archivos estáticos de forma masiva y asíncrona, liberando esa carga de Flask.

6. **¿Qué es un proceso “no-root” en un contenedor y por qué importa en seguridad? ¿Su imagen debería tener un usuario dedicado?**

Un proceso “no-root” significa que la aplicación se ejecuta bajo un usuario estándar limitado, en lugar del administrador del sistema (**root**).

Es crucial por seguridad porque si un atacante logra comprometer la aplicación Flask, sus acciones quedarán restringidas dentro del contenedor, evitando que tome el control del Sistema Operativo del anfitrión (*host*) mediante un escape de contenedor. Por lo tanto, la imagen incluirá obligatoriamente un usuario dedicado.

2. Diseño del Dockerfile

El diseño del contenedor se enfocó en garantizar la portabilidad del sistema, la seguridad mediante un usuario no root y el aprovechamiento del sistema de Docker.

2.1. Archivo de Exclusiones (.dockerignore)

Para evitar la transferencia de archivos innecesarios o locales al contexto de construcción de Docker (lo que incrementaría el tamaño de la imagen y sobrescribiría datos limpios de producción), se definió el siguiente archivo de exclusiones en la raíz del proyecto:

Listing 1: .dockerignore implementado

```
1 .venv/  
2 __pycache__/  
3 *.pyc  
4 *.pyo  
5 *.pyd  
6 instance/  
7 *.db  
8 *.sqlite3  
9 .DS_Store  
10 __MACOSX/  
11 .git/  
12 .gitignore
```

2.2. Implementación del Dockerfile

Listing 2: Dockerfile personalizado para NoticeBoard

```
1 FROM python:3.11  
2  
3 RUN useradd -m usuario  
4  
5 WORKDIR /app  
6  
7 COPY requirements.txt .  
8  
9 RUN pip install -r requirements.txt  
10  
11 COPY . .  
12
```

```

13 RUN chown -R usuario:usuario /app
14
15 USER usuario
16
17 ENV PORT=5000 \
18     FLASK_DEBUG=0 \
19     PYTHONUNBUFFERED=1
20
21 EXPOSE 5000
22
23 CMD ["python", "run.py"]

```

2.3. Decisiones de Diseño Justificadas

- **Selección de la Imagen Base (python:3.11):** Se optó por utilizar la imagen oficial completa de Python 3.11. Esta decisión de diseño garantiza una compatibilidad absoluta con cualquier dependencia porque fue desarrollado con esta version de base.
- **Optimización del Flujo de Capas y Reutilización de Caché:** Se estructuró el archivo separando la copia del archivo `requirements.txt` del resto del código fuente de la aplicación (`COPY . .`). Debido a que asegura que, si se realizan modificaciones futuras exclusivamente en las plantillas HTML, estilos CSS o lógica de rutas en Python, Docker mantendrá intacta y reutilizará la capa correspondiente al `pip install`. Esto evita la descarga e instalación repetitiva de paquetes, mejorando los tiempos de compilación.
- **Aislamiento Operativo y Mitigación de Riesgos (No-Root):** Se implementó la creación del usuario del sistema denominado `usuario` mediante el comando `RUN useradd. USER usuario`, se restringe el proceso del servidor Flask para que se ejecute bajo un entorno con privilegios mínimos. Esta configuración es vital para la seguridad en entornos distribuidos, impidiendo que un atacante tome el control del sistema anfitrión (*host*) en caso de que logre comprometer la aplicación web.

2.4. Cuestionario de Conceptos de Contenedores

1. **¿Cuál es la diferencia entre CMD y ENTRYPOINT? ¿Cuál conviene aquí y por qué?**

Ambas instrucciones definen el proceso principal que se ejecutará al iniciar el contenedor. La diferencia radica en que `ENTRYPOINT` establece un ejecutable fijo que no se sobrescribe fácilmente, tratando los argumentos pasados en el comando `docker run` como parámetros adicionales del script. Por el contrario, `CMD` define un comando por defecto que puede ser completamente reemplazado en tiempo de ejecución de manera directa (por ejemplo, pasando `/bin/bash` para abrir una consola interactiva). En este proyecto conviene utilizar `CMD`, ya que otorga mayor flexibilidad para realizar tareas de depuración, mantenimiento y pruebas dentro del contenedor sin alterar la configuración base.

2. **¿Qué hace exactamente la instrucción EXPOSE? ¿Expone el puerto realmente o es solo documentación?**

La instrucción `EXPOSE` funciona **estrictamente como documentación**. Simplemente actúa como un metadato informativo para los administradores de sistemas indicando en qué puerto interno espera recibir tráfico la aplicación Flask (puerto 5000).

3. ¿Qué es `.dockerignore` y qué archivos debería excluir en este proyecto?

Es un archivo que le indica a Docker qué archivos o carpetas locales debe ignorar al momento de construir la imagen, evitando copiar elementos innecesarios al contenedor.

En este proyecto de NoticeBoard, se excluyen específicamente:

- **Entornos virtuales:** `.venv/`
- **Caché compilada de Python:** `__pycache__/, *.pyc`
- **Base de datos local:** Directorio `instance/` y archivos `*.db` o `*.sqlite3`
- **Archivos del sistema y repositorios:** `.DS_Store`, `__MACOSX/` y la carpeta `.git/`

Se excluyen para mantener la imagen final lo más ligera posible, evitar conflictos entre las librerías de la computadora anfitrión y las del contenedor, y asegurar que no se filtren datos locales de prueba a un entorno de producción.

4. ¿Por qué es mala idea usar la etiqueta 'latest' como imagen base en producción?

Utilizar la etiqueta `latest` es una mala práctica debido a que en entornos productivos rompe el principio de **inmutabilidad y repetibilidad**. Dicha etiqueta apunta dinámicamente a la última versión disponible publicada por los mantenedores. Si la imagen del contenedor se reconstruye en el futuro, Docker descargará silenciosamente una versión de Python más reciente, lo que introduce un alto riesgo de romper la compatibilidad con las librerías declaradas en el proyecto y provocar fallos inesperados en producción. Definir una versión semántica estricta (como `python:3.11`) garantiza un comportamiento determinista.

3. Construcción y Verificación

Pasemos a la creación de la imagen de noticeboard

3.1. Bitácora de Construcción y Verificación Local

La compilación de la imagen se realizó de manera exitosa en la raíz del proyecto utilizando una sintaxis de Docker Hub. El comando ejecutado fue:

```
1 docker build -t daft4less/noticeboard:v1.0.0 .
```

El motor de Docker procesó las directivas del archivo de configuración, empaquetando el entorno de ejecución de Python 3.11 junto con las dependencias aisladas del sistema.

Una vez finalizada la compilación, se ejecutó el comando `docker images` para listar el inventario local y confirmar que la estructura de la imagen se almacenó con el nombre de repositorio y la etiqueta de versión semántica correctas.

3.2. Evidencias de Verificación Funcional

Para validar la portabilidad y la inyección dinámica de configuraciones desde el entorno externo, el contenedor se desplegó mapeando el puerto alternativo del host (8080) al puerto nativo interno (5000) mediante la siguiente instrucción:

```
1 docker run -d -p 8080:5000 --name contenedor_noticeboard -e PORT
  =5000 daft4less/noticeboard:v1.0.0
```

3.3. Análisis Técnico de la Imagen (Capas y Dimensiones)

Al ejecutar la inspección del historial mediante la directiva `docker history`, se determinó la estructura interna de la imagen construida:

- **Capa más pesada:** La capa generada por la instrucción `FROM python:3.11` representa la base del almacenamiento del contenedor (aproximadamente 1 GB debido al uso de la distribución Debian completa). La segunda capa con mayor impacto corresponde al comando `RUN pip install`, que añade el peso neto de las librerías web instaladas.
- **Comparativa de Crecimiento:** Al comparar la imagen resultante contra la imagen base de Python, el tamaño final se incrementó únicamente por el peso de los paquetes descargados por `pip` y el código fuente (`app/`, `config.py`, etc.), demostrando que el archivo `.dockerignore` evitó con éxito el traspaso de datos residuales de desarrollo (como el entorno virtual local de más de 40 MB).
- **Estrategias de Reducción Futuras:** Si fuera mandatorio optimizar y compactar drásticamente el tamaño de la imagen en disco, se debería modificar la directiva inicial del `Dockerfile` para migrar hacia una imagen base reducida como `python:3.11-slim`, o concatenar los comandos de instalación y limpieza en una única línea de ejecución para evitar la persistencia de subcapas redundantes.

3.4. Comando para Despliegue en Nuevos Entornos

Para que cualquier miembro del equipo de desarrollo pueda levantar la aplicación instantáneamente en una máquina nueva, debe ejecutar la siguiente instrucción en su terminal:

```
1 docker run -d -p 8080:5000 --name noticeboard_app -e PORT=5000
   daft4less/noticeboard:v1.0.0
```

Justificación de los argumentos utilizados:

- `-d`: Para que el contenedor corra en segundo plano y no deje bloqueada la terminal.
- `-p 8080:5000`: Para conectar el puerto 8080 de nuestra computadora con el puerto 5000 del contenedor (que es donde corre Flask).
- `--name noticeboard_app`: Doy un identificador al contenedor para facilitar los comandos posteriores de administración y monitoreo de logs.
- `-e PORT=5000`: Para mandar la variable de entorno al código y decirle en qué puerto tiene que arrancar.
- `daft4less/noticeboard:v1.0.0`: Es el nombre exacto de la imagen y la versión que queremos descargar y levantar.

4. Publicación

PAsumos a la publicacion publica para que cualquier persona pueda descargar y correr la aplicación fácilmente desde cualquier computadora, subimos la imagen final a nuestro repositorio en Docker Hub.

4.1. Enlace al Repositorio Público

La imagen se encuentra disponible de forma pública para la comunidad técnica en la siguiente dirección URL:

<https://hub.docker.com/r/daft4less/noticeboard>

4.2. Verificación del Despliegue desde el Registry

Para verificar la portabilidad absoluta, se procedió a iniciar sesión en Docker Hub y subir la imagen. Posteriormente, se eliminó la copia de la imagen local en la máquina de desarrollo utilizando el comando `docker rmi`. Finalmente, se ejecutó la inicialización del contenedor invocando la imagen directamente desde el registro remoto sin disponer de ningún código fuente local. El contenedor descargó las capas de internet y levantó el servicio de manera idéntica.

4.3. Análisis de Versionado y Seguridad en Producción

1. **¿Qué pasa si el registry tiene una versión 1.0.0 y usted sube una corrección de bug con la misma etiqueta? ¿Cómo manejaría el versionado?**

Si se sube una corrección de error utilizando la misma etiqueta (`v1.0.0`), Docker Hub sobrescribirá la imagen anterior apuntando el tag al nuevo commit de compilación. Esta es una mala práctica, ya que rompe la inmutabilidad: otros servidores que hagan un `docker pull` de la versión modificada obtendrán códigos distintos. Para manejar correctamente el versionado, se debe aplicar el estándar de **Versionado Semántico (SemVer)**. La corrección de un bug requiere incrementar el dígito de *patch*, publicando la actualización bajo la etiqueta `v1.0.1`.

2. **¿Qué es el digest de una imagen y para qué sirve en un contexto de seguridad?**

El *Image Digest* es un identificador único generado mediante un algoritmo que calcula la huella digital exacta del contenido de la imagen y todas sus capas. A diferencia de las etiquetas (tags) que pueden alterarse, el *digest* es **estrictamente inmutable**. En contextos de alta seguridad, desplegar contenedores llamando directamente a su digest (ej. `repo/imagen@sha256:xxxx...`) garantiza de manera matemática que el código ejecutado en producción no ha sido manipulado, inyectado con malware o modificado maliciosamente en el registro.

5. Uso de Inteligencia Artificial

En el desarrollo de esta actividad use la IA como soporte para el desarrollo de la actividad aquí detallare en que exactamente:

1. **¿Qué herramienta(s) usó?**

Use el modelo de Gemini como asistente en dudas planteadas a lo largo de la actividad como también problemas internos a la hora de usar python más precisamente el entorno virtual.

2. **¿En qué momento(s) específico(s) del proceso?**

Se implementó a la hora de investigar las preguntas planteadas en los entregables del proyecto, como también a la hora del despliegue de la aplicación porque en cmd daba problemas con la ruta de python

3. **¿Qué le dio la IA y qué tuvo que agregar o corregir usted?**

La IA me entregó buenos ejemplos de código, pero a veces eran demasiado exagerados

o complejos para esta práctica. Así que mi trabajo consistió más en reducir el código y a su vez reescribir las librerías base porque la IA incluía librerías extra que metía por su cuenta; porque aunque en teoría eran útiles, terminaban causando problemas de incompatibilidad con nuestro sistema base.

4. **¿Hubo algún momento en que la IA le dio algo incorrecto? ¿Cómo se dio cuenta?**

Sí, mas que dar errores fatales, la IA a veces generaba cosas que no tenían relación con nuestro trabajo. Por ejemplo, generaba recomendaciones para el `.dockerignore` larguísimo bloqueando carpetas y extensiones de archivos que ni siquiera existen en nuestro proyecto. Y, como mencioné arriba, también quería forzar el uso de versiones más nuevas de las librerías en lugar de respetar las versiones exactas con las que la app ya funciona. Me di cuenta de esto porque al interactuar con la estructura original como se mencionaba en el inicio de las directrices del trabajo jugué con parámetros del código por lo que sabía de antemano que contenían nuestras carpetas y lo que realmente pide nuestro `requirements.txt`.